Filler tokens in open models: a screen, a training attempt, and what a dot position is doing

Nick Rui
with the DeepSeek V4 Flash results and pipeline by Nicole (the `extract_dsv4.py` line of work)

September 3, 2026

Summary

Nicole found that DeepSeek V4 Flash solves a two-step variable-binding puzzle 70% of the time, and 98% of the time if fifty dots are inserted between the question and the answer. Dots placed before the question do nothing. Her lens and patching work showed the model staging intermediate values in the dot positions, copying them across dots, and reading them back late. The obvious question was whether smaller, open models do the same. They do not, and this report documents how thoroughly they do not.

I ran the same items through ten open models from 4B to 35B parameters (Qwen3, Qwen3.5, Qwen3.6, Llama-3.1, Gemma-3, OLMo-3.1; dense and mixture-of-experts). Every one scores between 0 and 14% on the two-step items at every dot count, with no placement-specific gain. On an easier one-step variant where they score 50 to 100%, dots still change nothing. Sampling more answers would not help either: the expected pass@20 barely moves with dots.

I then fine-tuned Qwen3.5-9B with LoRA to solve the task with dots present. Five variants (mixed dot counts, a harder chain, a 4B model, dots-only training, no-dots training) all learn the task, and all learn it as a direct computation at the answer position. The model trained *only* on prompts with dots scores 40/50 with no dots at all. On held-out items, its dot positions decode to the dot token, carry no stage value beyond a faint late leak, can be wiped at every block without changing the answer (median shift 0.02 nats), and receive 1 to 4% of the answer position's attention. Fine-tuning with dots made the dots more uniform and less attended than in the untrained model. The model learned to route around them.

Running the same anatomy on DeepSeek V4 Flash itself (Part V) shows the mirror image: its answer positions put 16 to 43% of late-block attention on the dots, the dot residuals are problem-specific and split by hyper-connection stream, and the answer is decodable from the dots but not from the question token. Running the released DeepSeek-V4-Flash-Base checkpoint, the same network before post-training, settles where this comes from (Part VI): the base scores 48/50 with no dots and 50/50 with fifty, in either placement, in chat and raw-text rendering. The chat model's 70% is a deficit that post-training introduced, and the dots recover the pretrained competence. The base also reads the dots exactly as the chat model does (20% of late-block attention, problem-specific dot residuals, the answer decodable from the span), so that anatomy is pretrained. What post-training changed is the direct path: the answer is readable from the last question token at R^2 0.74 in the base and 0.40 in chat. Reading trailing tokens is a pretrained habit of this architecture; needing them is what post-training added. Part VII shows the trigger: demonstrations that show a filler span, with no filler delivered in the target, move the answer off the chat model's question token in proportion to the span they show (0.84 with nothing announced, 0.51 for five, 0.31 for

fifty), while the base reacts less (0.55 at fifty); the instruction sentence that announces the span costs the chat model accuracy without moving the computation, and does nothing to the base. Numbers work as filler as well as dots and letters work less well, in any order, and attention on the filler tracks the gain; adjacent-position cosine along the span shows no point where the model changes thought, only tokenization artifacts.

Contents

1 Background

1.1 The claim being tested

The filler-token paper (arXiv:2607.03502) reports that a model's accuracy on some multi-step tasks rises when meaningless tokens are inserted between the question and the answer. The tokens carry no information. If they help, the model must be doing computation in those positions during the forward pass, computation it could not fit into the positions it already had.

The task used throughout is chained variable binding. A prompt lists five definitions, some literal and some referring to an earlier variable, and asks a question that requires resolving one reference. An example from the released set:

```
nof = 97
hoz = twice the number for nof minus 21
Question:  What is twice the number for hoz plus 28?
```

The answer needs two hidden steps: $97 \to 194 \to 173$, then $173 \to 346 \to 374$. Neither intermediate appears in the prompt. The scaffold is five demonstrations followed by the target, with a system sentence stating the exact number of filler tokens, and a line `Filler: . . . . .` between the question and `Answer:`. I kept this scaffold unchanged for every model, including the fine-tuned ones.

1.2 What Nicole found on DeepSeek V4 Flash

On the first 50 released items: 35/50 correct without dots, 49/50 with fifty dots after the question, 35/50 with fifty dots before the question (14 items rescued, none hurt, exact paired $p \approx 1.2 \times 10^{-4}$). The result replicated on 100 further items (59% to 94%). Her readouts showed the base value, the hidden bound value, the second product, and the answer becoming decodable in the dot positions

in that order by *depth* (layers 25 to 36), scattered across non-adjacent dots. Single-cell activation patching on an exact-layout counterfactual pair moved the counterfactual answer by up to 4.9 nats from one dot cell, and sixteen-cell doses took it from rank 82 to rank 2. Cross-position transplants showed the content is portable and the late dots are preferred receivers.

She also found that a Jacobian lens (a fitted linear map from each layer to the final layer) exposed nothing the plain logit lens did not. On her advice I used only the logit lens here.

1.3 What this report adds

Three things. A behavioral screen of ten open models on the same items. An attempt to train a model that uses the dots, with a matched design that can tell dot use apart from generic learning. And a mechanistic account of what a dot position contains and does in the trained model, using lens grids, lesions, patching, linear probes, and attention.

2 Setup

2.1 Harness

`scripts/extract_hf.py` is a single-GPU port of Nicole's behavioral sweep to Hugging Face checkpoints. It reuses her prompt builders, config format, and output schema, so her analysis scripts run on its output unchanged. Prompts go through each model's own chat template with thinking disabled. Decoding is greedy, three tokens. Every run records the rendered prompt, token ids, the alignment of visible dots to tokens, the generated text, and the full first-step logits.

Checks I ran before trusting any number: the dot count equals the filler token count at every dot count for every tokenizer (the last Llama dot merges with the following newline, exactly as on DeepSeek; Qwen dots do not merge); greedy runs are bit-identical across repeats; cached and uncached logits differ by at most 0.125 in bfloat16 with the same argmax; plain prompts get sensible answers; and Nicole's prompt tests pass on the compute box.

2.2 Metrics

Accuracy is greedy exact match. For each dot count $k > 0$ I report helped and hurt counts against $k = 0$ and an exact McNemar test on the discordant pairs. The sensitive endpoint is the teacher-forced log-probability of the correct answer, because accuracy flips are rare on confident prompts. From that log-probability I compute expected pass@n at temperature 1 as $1 - (1 - p)^n$ averaged over items, which is what sampling n times would converge to.

One tokenizer fact shapes everything downstream. Qwen and Gemma split numbers into single digits, so `219` is three tokens. Llama keeps up-to-three-digit numbers whole. Where a rank is reported for a Qwen model it is the rank of the answer's first digit.

2.3 Hardware

One A100 40 GB for the small models, then one H100 80 GB for everything else. Every 30B-class model ran in bfloat16 on the H100 without quantization. Runs were 3 to 20 seconds per item. The DeepSeek runs (Parts V and VI) used a 4×H100 80 GB box with Nicole's 4-way model-parallel loader: the chat checkpoint (FP8 attention, FP4 experts) takes 42 GB per GPU and the base (FP8 experts) 72 GB.

3 Part I: ten open models show no filler effect

3.1 The released two-step items

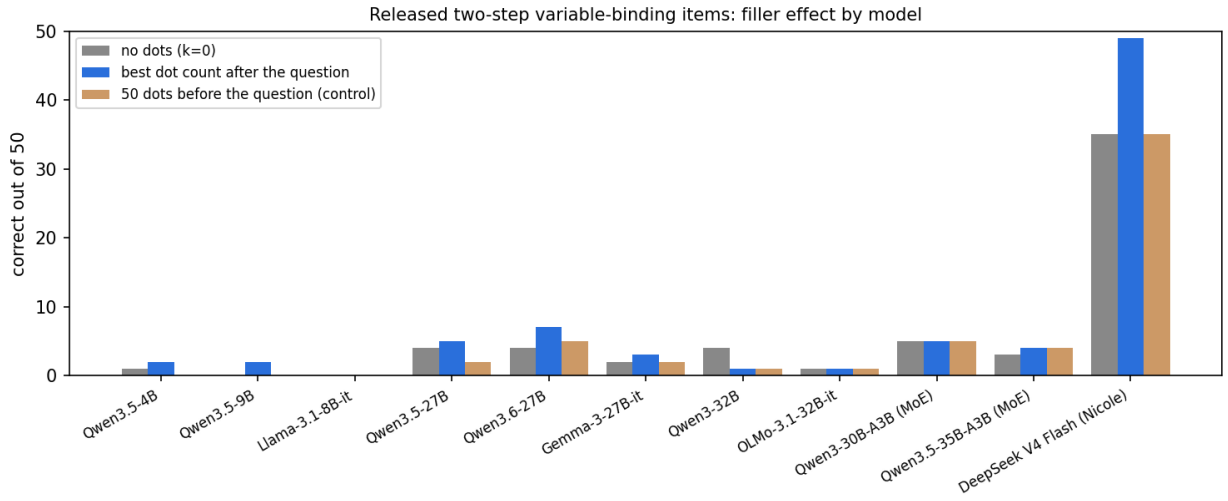

Figure 1: Correct out of 50 released two-step items for each model without dots, at the best dot count after the question, and with fifty dots before the question. DeepSeek V4 Flash (Nicole's result) is on the right for comparison.

Model	Type	$k = 0$	best dots	control	smallest p
DeepSeek V4 Flash (Nicole)	MoE, 4-stream residual	35	49	35	1.2×10^{-4}
Qwen3.5-4B	dense	1	2	0	1
Qwen3.5-9B	dense	0	2	0	0.5
Llama-3.1-8B-Instruct	dense	0	0	0	1
Qwen3.5-27B	dense	4	5	2	1
Qwen3.6-27B	dense	4	7	5	0.375
Qwen3-32B	dense	4	1	1	0.125 (dots hurt)
Gemma-3-27B-IT	dense	2	3	2	1
OLMo-3.1-32B-Instruct	dense	1	1	1	1
Qwen3-30B-A3B	MoE, 3B active	5	5	5	1
Qwen3.5-35B-A3B	MoE, 3B active	3	4	4	1

Table 1: Released two-step items, correct out of 50. "Control" is fifty dots placed before the definitions and question. The last column is the smallest exact McNemar p across the five dot counts.

Every model is at floor. The best movement anywhere is Qwen3.6-27B going from 4 to 7 at two dot counts (4 helped, 1 hurt, $p = 0.375$, one of five lengths tested). Qwen3-32B and Qwen3-30B-A3B get worse with dots, in both placements.

The failures are not noise. Outputs are clean numbers. The smaller models emit a wrong number of roughly the right size; the 27B models get close (Qwen3.5-27B has a median error of 8 to 10 and more than half its answers within 10 of correct) without being right. Dots change the emitted answer in 80 to 90% of items for every model. They perturb without steering.

Two facts show the failure is about direct-answer mode, not capability. With chain-of-thought enabled, Qwen3.5-9B solves 3/3, Llama 4/5, Qwen3.5-27B 5/5. And the plain arithmetic "compute

$2 \cdot 97 - 21$, then twice that plus 28" gets `186` from the 9B and `370` from the 27B (correct: 374). The models cannot do two dependent steps in one forward pass at these sizes, so there is no partial circuit for filler to help.

3.2 Log-probability and pass@n

Accuracy on 50 items is coarse. The paired log-probability of the correct answer is not, and it agrees.

Model	best $\Delta \log p$ after question	$\Delta \log p$, control	E[pass@20], $k = 0 \to$ best
Qwen3.5-27B	+0.08 (27/23 items up/down, $p = 0.32$)	−0.09	0.47 → 0.50
Qwen3.5-35B-A3B	+0.00	+0.33	0.20 → 0.21
Qwen3.6-27B	+0.13 ($p = 0.67$)	−0.05	0.51 → 0.52
Qwen3-32B	+0.95 ($p = 0.67$)	+0.05	0.20 → 0.26
Qwen3-30B-A3B	−0.12	−0.11	0.23 → 0.23
Gemma-3-27B-IT	+1.22 ($p = 0.20$)	+3.08 (40/10)	0.04 → 0.06
OLMo-3.1-32B	+1.25 (36/14, $p = 0.003$)	+0.75	0.10 → 0.14

Table 2: Mean change in $\log p$(correct) versus $k = 0$, best dot count, against the pre-question control at $k = 50$.

Gemma's gain with dots is smaller than its gain with dots before the question, so it is a context-length effect. OLMo at $k = 100$ is the only placement-specific cell in the whole screen, about half a nat net, on an answer whose median rank is in the fifties. Nothing here would survive sampling: pass@20 stays between 0.04 and 0.52 and moves by a point or two.

3.3 An easier task does not help

The released items are all one hidden hop before the final operation. I generated a one-step variant (`scripts/build_onestep_varbind_configs.py`): same scaffold and vocabulary, five definitions with derived-expression distractors, but the queried variable is a literal, so the answer needs one hidden step. This puts the small models at 50 to 85% and the 30B models at 82 to 100%. Dots still do nothing. Qwen3.5-4B showed a 37 to 42 bump at five dots on 50 items (6 helped, 1 hurt, $p = 0.125$); on 200 fresh items it was 147 to 145, with helped and hurt within two of each other at every dot count, and the pre-question control slightly negative (4 helped, 12 hurt). That bump was noise.

4 Part II: training a model to use the dots

Since no open model does this out of the box, the next step was to make one. The design question is how to tell "the model uses the dots" apart from "the model learned the task." The answer is to train with a mix of dot counts, including zero, and watch whether accuracy at $k = 0$ stays behind accuracy with dots. DeepSeek looks like that: 70% at $k = 0$, 98% at $k = 50$.

4.1 Data and recipe

4,000 synthetic chain-length-1 items (`scripts/build_varbind_sft_data.py`) matched to the released distribution: one to four literals, all coefficients "twice," constants 1 to 30, derived expressions pointing at another derived variable 37% of the time, and every released item excluded by signature. Each item is rendered once per epoch at a dot count drawn from $\{0, 5, 10, 25, 50, 100\}$. Only the answer tokens and the end-of-turn token carry loss; the prompt, demonstrations, and dots are

context. LoRA rank 32 on all linear layers, learning rate 10^{-4} with cosine decay, effective batch 16, 500 optimizer steps (two epochs). Every 100 steps the adapter is evaluated on the released 50 items at every dot count and on the pre-question control, with the same code as the screen. Records are in `results/qwen3.5-9b/lora-*/` and `results/qwen3.5-4b/lora-mixedk/`.

4.2 Five runs, one outcome

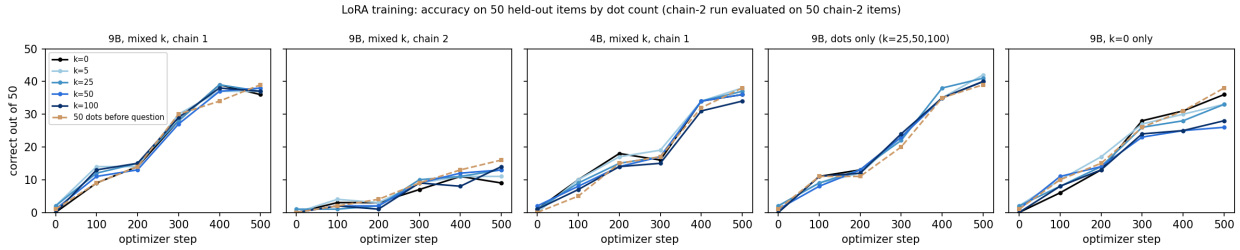

Figure 2: Accuracy every 100 steps for the five training runs. Solid lines are dot counts after the question; the dashed line is fifty dots before the question. A model that used dots as a workspace would show the $k = 0$ line staying below the others. None does.

Run	trained on	$k = 0$	$k = 50$	$k = 100$	control
Qwen3.5-9B, chain 1	$k \in \{0, 5, 10, 25, 50, 100\}$	36	38	37	39
Qwen3.5-9B, chain 2	mixed k (harder task)	9	13	14	16
Qwen3.5-4B, chain 1	mixed k	36	36	34	38
Qwen3.5-9B, chain 1	dots only, $k \in \{25, 50, 100\}$	**40**	40	40	39
Qwen3.5-9B, chain 1	$k = 0$ only	36	**26**	28	38

Table 3: Correct out of 50 at step 500. The chain-2 run is evaluated on 50 held-out chain-2 items.

The mixed-k run learns the task (0 to 72%) and learns it uniformly. The only moment dots led was step 100 (9 at $k = 0$ versus 11 to 14 with dots, control 9), and it closed by step 200. A 9B model fits the two-step chain in one forward pass, so mixed supervision gives it no reason to route through the dots.

I tried making the direct computation not fit by adding a hop (chain length 2, three sequential affine steps). The model learns slowly (18 to 28% at step 500) and the only dot effect is again larger with dots *before* the question (16) than after (13 to 14). I tried a smaller model (Qwen3.5-4B) on the theory that it might not fit two steps. It matches the 9B exactly.

The paired design settles it. The model trained only on prompts with dots, which never saw a dotless prompt, scores 40/50 without dots. Whatever it learned lives outside the dot positions. The mirror image, the model trained only without dots, loses ten items when dots are inserted after the question (14 hurt, 4 helped at $k = 50$) and none when they are inserted before it. For an untrained-on-dots model the dots are interference at the answer position. Training with them present teaches the model to make them inert, not to use them.

5 Part III: is the trained model using the dots at all?

Everything in this section is on held-out items the trained model never saw and that were not used to pick a checkpoint. The model under study is the dots-only adapter, which is the strongest "good

with dots" model (80% on the released items, 67 to 73% on 100 held-out items at every dot count).

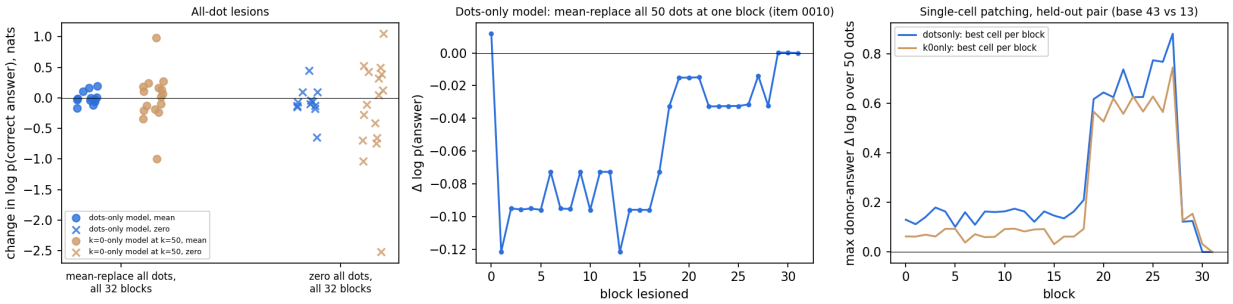

Figure 3: Left: change in $\log p$(answer) when every dot residual is replaced at all 32 blocks at once, per item. Middle: same replacement at one block at a time, one item. Right: single-cell patching of donor dot residuals into a target prompt, best cell per block, for a held-out pair differing in one prompt token.

5.1 Lesions: wipe every dot at every block

`scripts/patch_varbind_hf.py --lesion-all-dots --lesion-all-layers` replaces all 50 dot residuals with their mean over positions (or with zeros) at all 32 decoder blocks simultaneously, so no later block can re-read pre-lesion dot content. On twelve items the dots-only model gets right at $k = 50$, the mean lesion shifts $\log p$(answer) by a median of -0.02 nats (range -0.17 to $+0.20$) and changes one greedy output, to the correct answer. Zeroing everything shifts the median by -0.09 and leaves 11 of 12 correct.

The lesion has teeth. On the $k = 0$-only model at $k = 50$, where dots hurt behaviorally, the same intervention swings $\log p$ by up to ± 1 nat (mean) or ± 2.5 (zero), changes four to eight of sixteen greedy outputs, and repairs two to three wrong answers. Dot content is causally live in that model, as interference.

5.2 Patching: 1,600 single cells

From held-out item 0067 ($43 \to 99 \to 198 \to 203$) I built a one-digit counterfactual ($13 \to 39 \to 78 \to 83$) whose prompt differs at exactly one token. The dots-only model answers all 18 members of the family correctly except two near misses. Patching the donor's dot residual into the target one cell at a time over 32 blocks $\times$ 50 dots, the donor answer 83 starts at $\log p = -20.5$ and the best single cell adds 0.88 nats. The target answer never drops more than 0.03. Nicole's DeepSeek pair gave 4.9 nats from one cell. The only structure is a band at blocks 19 to 27 where the best cells reach 0.6 to 0.9 nats, matching the faint late leak in the lens grids below.

5.3 Logit-lens grids

Eight held-out items at $k = 50$ (the five the dots-only model gets right only with dots, plus three it gets right either way and whose stage values have distinct first digits), each with Nicole's stage-matched derangement controls, for the dots-only, $k = 0$-only, and base models. Grids cover all 32 blocks at all 50 dots plus the answer cue and generation position; final-block closure error is 0.0 everywhere.

The dot positions carry nothing stage-specific by this readout. Rank-1 hits for true stage digits in filler cells (at most 3 of 8 items, at most 0.9 of 1,600 cells) match the control labels. Averaged over late-third filler cells, the true answer digit beats its control by 0.12 nats in the dots-only

model and 0.36 in the $k = 0$-only model, and by about zero in early and middle blocks. The top-1 token at a filler cell is a digit in 0.2% of late cells; it is most often the dot token itself. At the generation position the answer's first digit is rank 1 at block 30 of 32 in 8/8 items. Viewers: `results/qwen3.5-9b/lens-heldout-k50/<model>/<item>/viewer.html`.

6 Part IV: what a dot position is doing

The lens can only see content along token directions. To see the rest I dumped every block's residual at all 50 dots, the last question token, the answer cue, and the generation position for 100 held-out items at $k = 50$, for the base, dots-only, and $k = 0$-only models, plus attention mass by prompt region in the eight full-attention blocks (`scripts/dump_dot_residuals_hf.py`, `scripts/analyze_dot_residuals.py`). Qwen3.5 alternates Gated DeltaNet blocks, which are linear attention with no attention matrix, with full attention; the attention results cover the eight full-attention blocks only.

6.1 A dot residual is almost entirely "I am dot number k"

Model	var. by position	var. by problem	cos, same dot, other problems
base	0.69 / 0.51 / 0.48	0.01 / 0.01 / 0.02	0.92 / 0.83 / 0.81
dots-only	0.88 / 0.74 / 0.60	0.01 / 0.02 / 0.05	0.99 / 0.99 / 0.97
$k = 0$-only	0.82 / 0.63 / 0.46	0.01 / 0.06 / 0.10	0.98 / 0.88 / 0.88

Table 4: At blocks 16 / 24 / 31: fraction of dot-residual variance explained by dot position versus by problem, and cosine similarity of the same dot position across different problems.

Which problem precedes the dots explains 1 to 5% of the dot residuals' variance in the dots-only model. The same dot position in two different problems has cosine 0.97 at the last block. Training with dots made the dots *more* problem-independent than in the base model (0.81 there); training without them left them less so (0.88) and gave problem identity its largest share (10%).

6.2 The small problem-dependent part is a selective copy of the relevant numbers

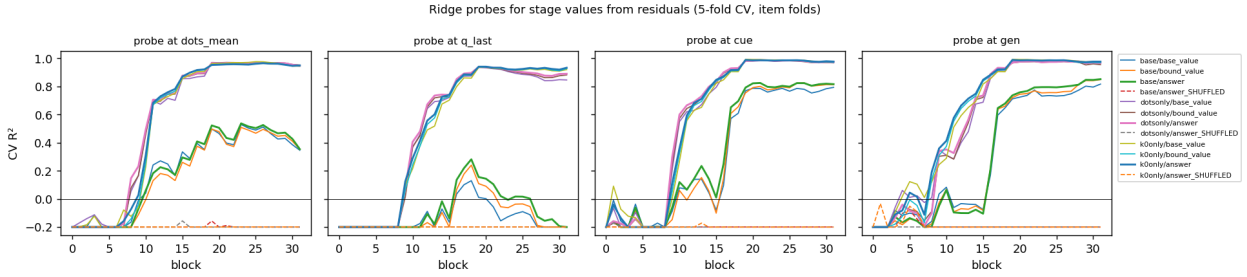

Figure 4: Ridge probes (5-fold cross-validation over items) predicting stage values from the mean dot residual, the last question token, the answer cue, and the generation position, by block. Dashed lines are shuffled-label controls.

Ridge probes read the queried chain's base literal from the mean dot residual at $R^2 = 0.98$ in the dots-only model (0.43 in the base), the chain's constants at 0.92 to 0.96, and the answer at 0.97.

A visible literal that is *not* on the queried chain is not decodable ($R^2 \leq 0.06$ after block 20). So the dots do reflect variable-binding resolution: they encode which numbers matter. Two qualifications. On an affine task the answer is a linear function of the inputs, so a linear probe cannot distinguish stored inputs from a computed answer, and "answer $R^2 = 0.97$" is not evidence the dots computed anything. And the same information sits at the answer cue at $R^2 = 0.99$ in every model. This is a faint redundant copy that nothing downstream reads, and fine-tuning sharpened it everywhere whether or not dots were in the training prompts (the $k = 0$-only model's dots reach the same R^2).

6.3 The answer positions do not look at the dots

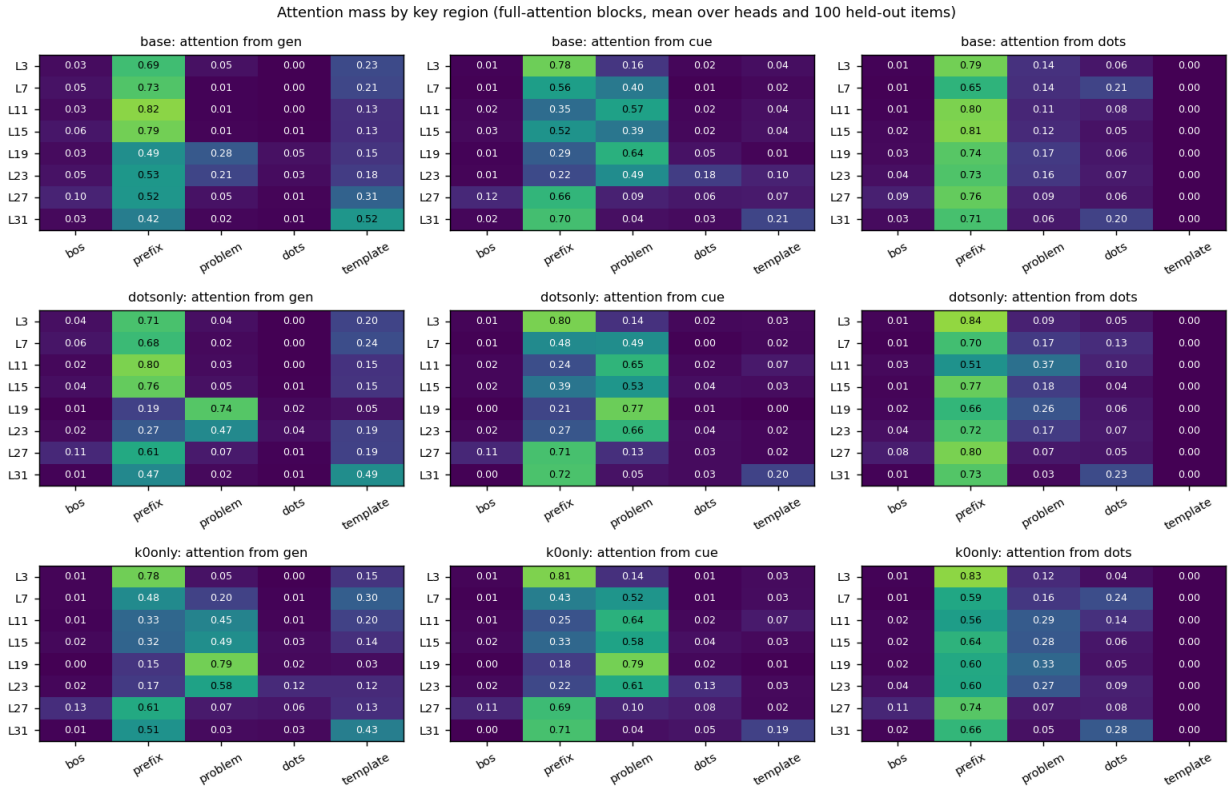

Figure 5: Attention mass by key region in the eight full-attention blocks, mean over heads and 100 items. Regions: BOS, prefix (system sentence and demonstrations), the target problem, the dots, and the answer template.

The dots-only model's answer cue and generation position put 1 to 4% of their attention on the 50 dots in every full-attention block. The base model's answer cue puts 18% there at block 23, and the $k = 0$-only model, which is hurt by dots, 12 to 13%. The single head most attentive to dots at the generation position reaches 0.08 in the dots-only model, 0.18 in the base, 0.24 in the $k = 0$-only model. Training with dots taught the answer positions to look away from them.

The dots themselves attend mostly to the few-shot demonstrations (50 to 84% of mass), then to each other (20 to 28% at the last block), and to the target problem only in the middle blocks (a 37% peak at block 11 in the dots-only model).

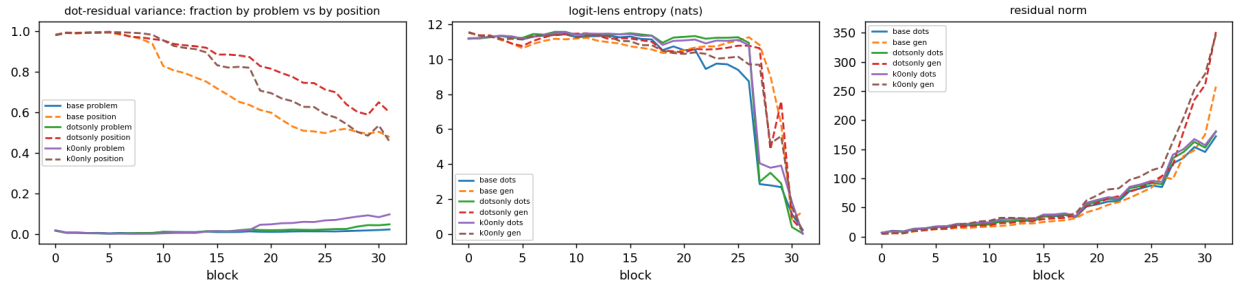

Figure 6: By block: fraction of dot-residual variance explained by problem versus by position; logit-lens entropy at dots and at the generation position; residual norm at dots and at the generation position.

6.4 The dots are processed, toward predicting the next dot

Logit-lens entropy at dot positions falls from 11.2 nats at block 0 to 0.02 nats at block 31 in the dots-only model, versus 0.14 at the generation position. Residual norms grow along the same curve as content tokens. Each dot spends its depth becoming certain that the next token is a dot. That is work, but it is not the task's work.

6.5 Summary of the anatomy

A dot position in the fine-tuned model holds a dominant position-identity vector, a faint linear copy of the queried chain's numbers that is present more strongly at the answer cue, attends to the demonstrations and to other dots rather than to the problem, receives 1 to 4% of the answer positions' attention, and predicts "dot." Fine-tuning with dots present made the dots more uniform and less attended than in the untrained model. This is the opposite of a workspace. It also explains why dots hurt the model trained without them: its answer cue attends to dot content three to four times more, and that content is out of distribution.

7 Part V: the same anatomy on DeepSeek V4 Flash

With a 4×H100 box I reran the dot-position analysis on the model that shows the behavioral effect, using Nicole's converted checkpoint and loader (her sanity gate passed: final-head closure and the lens identity anchor are exact). Fifty held-out items at $k = 50$; every block's raw four-stream residual at all 50 dots, the last question token, the answer cue, and the generation position; and attention recomputed from q, k, the indexer's selection, and the per-head sink at every block, because the fused sparse-attention kernel returns no weights (`scripts/dump_dot_residuals_dsv4.py`, `scripts/dump_dot_attention_dsv4.py`).

Two architecture facts frame the attention numbers. Every V4 block sees a 128-token sliding window of raw keys plus compressed keys: 4-token blocks chosen by a learned indexer in even blocks 2 to 40, 128-token blocks in odd blocks 3 to 41; blocks 0, 1, and 42 are window-only. At our prompt lengths the indexer's top-512 is every block, so sparsity never bites. For the answer positions the dots are about 40% of the raw window keys, and the demonstrations and most of the problem are reachable only through compressed keys.

Model	gen → dots	cue → dots	dots → dots	max head, gen → dots
Qwen3.5-9B base	0.010	0.046	0.13	0.18
Qwen3.5-9B dots-only	0.009	0.029	0.14	0.09
Qwen3.5-9B $k = 0$-only	0.043	0.067	0.18	0.24
DeepSeek V4 Flash	**0.164**	**0.195**	**0.26**	**0.90**

Table 5: Mean attention mass on the dot region over the last third of attention blocks, and the single most dot-attentive head at the generation position.

7.1 DeepSeek's answer positions look at the dots

DeepSeek puts 20% or more of the generation position's attention on the dots in blocks 21, 22, 24, 35, 39, and 40 (peak 0.35 at block 40), and of the answer cue's attention in fourteen blocks (peak 0.43 at block 39). Single heads reach 0.5 to 0.9 of their mass on dots in blocks 26 to 40. The dots attend to each other at 0.30 to 0.48 in blocks 33 to 42. The sliding window makes the dots cheap to reach, but Qwen had every token in reach and spent 1 to 4% on them; DeepSeek spends 16 to 43% in the blocks where Nicole's lens saw the ladder resolve.

7.2 The dots are problem-specific, and the four streams divide the work

Cosine of the same dot position across different problems at three-quarters depth is 0.78 in DeepSeek against 0.99 in the trained Qwen (0.83 in the Qwen base, 0.88 in the $k = 0$-only model); the problem explains 4 to 5% of late-block dot variance (Qwen dots-only: 2%). Per hyper-connection stream:

Stream	var. by problem (blocks 32 / 40)	cos, same dot, other problems	norm at dots, block 40
0	0.06 / 0.09	0.51 / 0.61	186
1	0.03 / 0.03	0.68 / 0.79	286
2	0.09 / 0.10	0.48 / 0.64	273
3	0.02 / 0.05	0.88 / 0.76	49

Table 6: DeepSeek V4 Flash dot residuals by hyper-connection stream.

Streams 0 and 2 carry the problem-specific content (cosine near 0.5, a tenth of their variance from problem identity). Stream 3 is the position-identity lane (cosine 0.88, small norm until the last blocks). Stream 1 is a high-norm carrier with little problem dependence. This is the first direct look at what the four lanes do at a filler position, and it fits the earlier guess that width lets a value be stored without overwriting the position's own state.

7.3 Where the answer is computed

Model	answer R^2 from dots (mean)	from last question token	from answer cue
Qwen3.5-9B dots-only	0.97	0.94	0.99
DeepSeek V4 Flash	0.87	**0.40**	0.87

Table 7: Best ridge-probe R^2 for the answer, by position.

In the trained Qwen the answer is already linearly present at the last question token, before any dot exists; the dots and the cue only repeat it. In DeepSeek the question token barely encodes it

and the dots encode it as well as the answer cue does, consistent with the computation happening across the dot span rather than at the question. The affine-probe caveat applies to the absolute values, not to this contrast.

One more difference. Logit-lens entropy at dot positions is 0.2 to 1.5 nats in blocks 0 to 18, jumps to 5.9 at block 19, and declines to 0 by block 42; Qwen's dots sit at 11 nats and only fall at the end. Whatever the early-block readout means under the stream collapse, the discontinuity at block 19 coincides with the first blocks where attention to dots exceeds 20%.

7.4 Summary

On the model that shows the behavioral effect, the dot positions are attended (16 to 43% late-block mass from the answer positions, single heads up to 0.9), problem-specific (cosine 0.78 across problems versus 0.99 in the trained Qwen), organized by stream (two content lanes, one position lane, one carrier), and hold the answer as strongly as the cue does while the question token does not. Every one of those is the opposite of what the fine-tuned Qwen showed. Whether post-training installed this is the subject of Part VI.

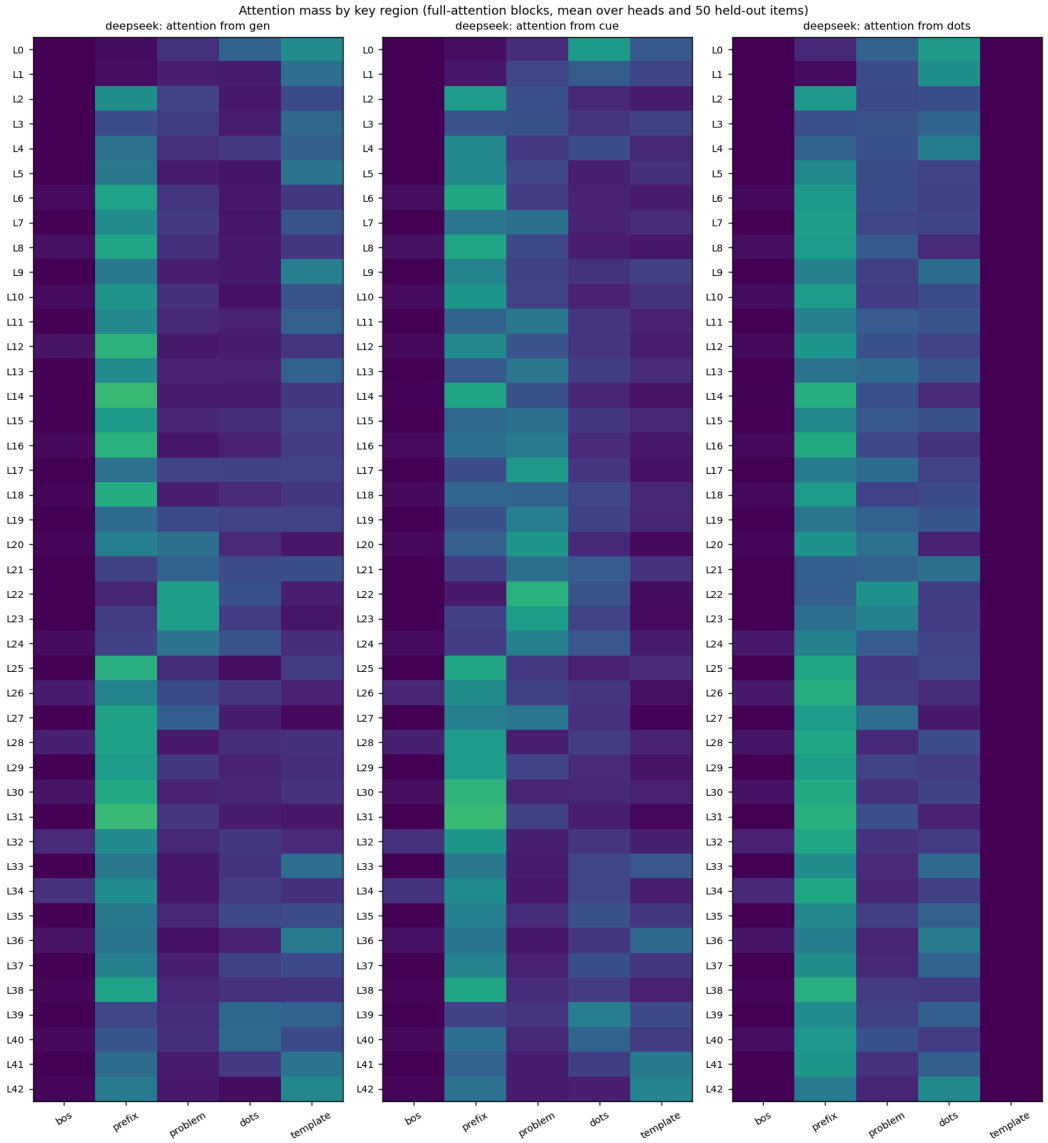

Figure 7: DeepSeek V4 Flash: attention mass by key region at every block, mean over 64 heads and 50 held-out items, from the generation position, the answer cue, and the dots.

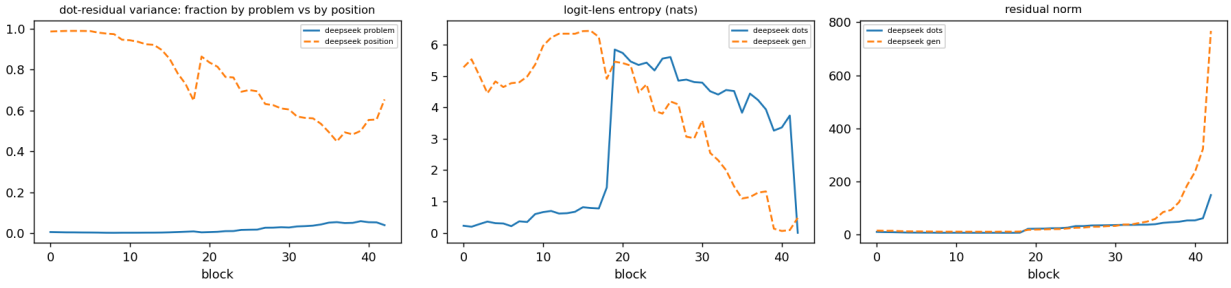

Figure 8: DeepSeek V4 Flash by block: fraction of dot-residual variance by problem versus position; logit-lens entropy (through Nicole's stream collapse) at dots and at the generation position; residual norm.

8 Part VI: the base checkpoint

`deepseek-ai/DeepSeek-V4-Flash-Base` has the same architecture and pretraining as the chat model and no post-training, so it separates the two remaining explanations. Its released experts are FP8 where the chat checkpoint ships FP4, which makes the converted 4-way shards 75 GB each against 42 GB. They fit on the 4×H100 box only with PyTorch's expandable-segments allocator: under the default caching allocator every 8 MiB expert weight is rounded into a 20 MiB segment and about 12 GiB per GPU is lost, which is exactly the margin. Loaded through the chat repository's reference code with `expert_dtype` unset, the sanity gate (final-head closure, lens identity anchor) passed with no failed prompts. Same 50 released items, same sequence length, same config hash as the chat run (`scripts/run_dsv4_base_eval.sh`, results under `results/deepseek-v4-flash-base/`).

A base model has no chat template of its own, so I ran two renderings. "Chat" is Nicole's encoding with turn markers, identical to the chat-model run. "Plain" (`extract_dsv4.py --render plain`) joins the system text and the five demonstrations as raw text with the demonstration answers inline and no turn markers.

8.1 The base does not need the dots

Visible dots	Chat model (Nicole)	Base, chat rendering	Base, plain rendering
0	35	48	47
5	45	48	44
10	42	49	50
25	43	49	47
50	49	50	50
100	49	49	50

Table 8: Correct out of 50 on the released two-step items, post-question dots.

The chat model's change from $k = 0$ to $k = 50$ is 14 helped and 0 hurt. The base's is 2 helped and 0 hurt in chat rendering (exact McNemar $p = 0.5$) and 3 and 0 in plain rendering ($p = 0.25$); every other dot count moves one to three items in either direction. The pre-question $k = 50$ control, where the chat model fell back to 35/50, gives the base 49/50 in chat rendering (2 helped, 1 hurt) and 50/50 in plain (3, 0). Placement does not matter to the base because there is no deficit for placement to fix.

One artifact to know about: the rank columns (MRR, R@10) in the plain-rendering reports are meaningless. The plain prompt ends in `Answer:` and DeepSeek's tokenizer emits a standalone space token before the number, which the base predicts with probability 0.997; the rank helper scores the number at that position. Accuracy is parsed from the generated text and is unaffected.

8.2 The base reads the dots the same way

The same dump as Part V on the base (`scripts/run_dsv4_base_dot_dump.sh`: 50 held-out items at $k = 50$, chat rendering, all four streams, recomputed attention), analyzed side by side with the chat run.

The base puts more of the generation position's attention on the dots than the chat model does, in the same blocks and with more dot-dominated heads. Its dot residuals are more problem-specific (cosine of the same dot across problems 0.64 at blocks 32 and 40 against 0.78 and 0.76 in chat), the answer is decodable from the dots at R^2 0.85 (chat 0.87), the entropy jump at the dots comes

Model	gen → dots	cue → dots	dots → dots	blocks, gen ≥ 0.20	blocks, head ≥ 0.5
V4 Flash (chat)	0.164	0.195	0.26	21, 22, 24, 35, 39, 40	26, 28, 32, 34, 37, 39, 40
V4 Flash Base	0.203	0.189	0.27	17–19, 21–24, 34, 35, 37, 39–41	21, 24, 28, 32–41

Table 9: Attention mass on the dot region over the last third of blocks, and where single heads put at least half their mass on dots.

at the same depth, and the four streams divide the work the same way (streams 0 and 2 content, stream 3 position, stream 1 carrier). Everything in Part V that looked like a workspace is already there before post-training.

8.3 What post-training changed is the direct path

Model	from dots (mean)	from last question token	from answer cue	from gen. position
V4 Flash Base	0.85	**0.74**	0.86	0.86
V4 Flash (chat)	0.87	**0.40**	0.87	0.87
Qwen3.5-9B dots-only	0.97	0.94	0.99	—

Table 10: Best ridge-probe R^2 for the answer, by position.

In the base the answer is linearly present at the last question token, before any dot exists, at 0.74. In the chat model that number is 0.40, while the dots and the cue hold it at 0.87 in both. Post-training weakened the encoding at the question and left the dot span as the place where the value is still assembled. That is the mechanistic counterpart of the behavioral table: the base can answer from the question alone (96% at $k = 0$); the chat model often cannot (70%), and the dot span, which both models read, recovers it (98%).

8.4 The chat model's misses are near-misses

What does the post-trained model emit when it fails at $k = 0$? All fifteen misses are numbers of the right magnitude; there is no refusal, no words, no formatting failure. None is a wrong-variable substitution: for every miss I resolved every variable in the prompt and applied the final operation to each, and no miss matches another variable's value or its transformed value. The model always finds the right chain. Nine of the fifteen are within 4 of the correct answer (seven within 2), one drops the doubling in the final step ($b + c$ for $2b + c$), one drops the constant ($2b$ for $2b + c$), and the rest are arithmetic slips of 10 to 50 on three-digit answers. With fifty dots, fourteen of the fifteen are repaired and the one remaining miss is off by 2.

The base misses two items at $k = 0$. One of them is the same dropped-constant error on the same item that the chat model makes, which suggests that item is hard for reasons shared by both checkpoints.

8.5 Relocated, not forgotten

"Post-training made the model forget the task" is too strong. Nothing about the task disappeared. With fifty dots the chat model reaches 49/50, the base's level, so the circuit exists and needs only positions after the question to run in. The answer is decodable from the dot span and the

answer cue at R^2 0.87 in both checkpoints, so the computation happens in both; in the base it also happens at the question token (0.74), in the chat model it mostly does not (0.40). And the misses are approximate answers to the right sub-problem, not wrong sub-problems. The accurate word is *relocated*. What weakened is one pathway: finishing the two-step arithmetic in place, at the question position, in time to emit the answer on the very next token. When any trailing tokens exist, even dots, the pretrained dot-reading anatomy carries the value to the answer and the pathway is not needed.

8.6 Why post-training would do this: hypotheses

None of these is tested here; they differ in what they predict, which is how to tell them apart.

Deferred computation. *Supported and graded (Part VII).* Reasoning-oriented reinforcement learning rewards putting the work into the span of tokens after the question. A model that has internalized "the span after the question is where thinking happens" has less pressure to keep a pathway that finishes arithmetic before the first output token, and unused pathways drift. Prediction: the deficit should be specific to direct-answer mode and should scale with how much post-training emphasized long reasoning; models with lighter or non-reasoning post-training from the same base should show a smaller deficit. This is the hypothesis the evidence fits best, and the awkward fact remains that Qwen3.5 also has a thinking mode and shows no filler gain; the difference would have to lie in how each lab trained its non-thinking mode.

A representational shift at the question token. *Resolved: it is deferral, not a shift (Part VII); with no filler announced the chat question token reads out at 0.84, like the base.* The 0.74 to 0.40 drop is a linear readout. Post-training may have moved the answer representation at the question token into a form a ridge probe cannot read, rather than removed it. Prediction: a nonlinear probe, or a patching experiment that transplants the chat model's question-token residual into the base, would recover more of the answer than the linear number suggests. The behavioral gap argues against this being the whole story, since whatever is at the question token is not enough to emit the right number.

Confidence recalibration. *Holds as a description (Part VII: entropy 0.76 versus 0.30 nats at $k = 0$, sharpened by dots).* Post-training may have made the model reluctant to commit to a computed number at the first output token, spreading probability across nearby values. The near-miss pattern (right magnitude, off by a few) is consistent with this. Prediction: the chat model's first-token distribution at $k = 0$ should be flatter across neighboring numbers than the base's, with the correct answer still ranked high. Nicole's MRR of 0.795 at $k = 0$ against 0.98 for the base says the correct answer is usually near the top even when it is not first, which fits.

Format-specific interference. *Refuted (Part VII: plain rendering drops the chat model to 19/50).* The turn markers and the empty think tag were seen during post-training in contexts where an answer follows a reasoning span. Their presence with no span may itself disrupt the direct path. Prediction: the chat model should do better at $k = 0$ in the plain rendering, without turn markers. The base did equally well in both renderings, but this was not run on the chat model and is a cheap check.

The first and third hypotheses are not exclusive; deferred computation would produce exactly the kind of imprecise first-token answers the third describes. Distinguishing them needs the chat

model's full first-token distributions at $k = 0$, which the existing sweep files contain, and a plain-rendering run of the chat model, which needs the box.

8.7 What this settles

The chat model's 70% at $k = 0$ is not a task the network cannot do; the same network before post-training does it at 96%. Fifty post-question dots bring the chat model to 98%, which is the base's level. So the filler effect in DeepSeek V4 Flash is a post-training artifact: post-training introduced a deficit in direct answering, and the dots recover the pretrained competence.

The anatomy splits the explanation in two. Reading the dots (16 to 43% of late-block attention, problem-specific dot residuals, the answer decodable from the dot span) is there in the base, so it comes from pretraining and the architecture, not from post-training; the sliding window that makes the dots 40% of the raw keys and the four-stream residual are the plausible reasons. What post-training added is the *dependence* on it: a weaker direct path at the question token (answer probe 0.74 to 0.40) and a 26-point $k = 0$ deficit that the dot span repairs. The "learned habit" reading was half right. The habit of reading trailing tokens is pretrained; needing them is learned. It reframes Nicole's result: the interesting object is what post-training did to the direct-answer path, and why the model's pretrained reading of an inert span is enough to restore it.

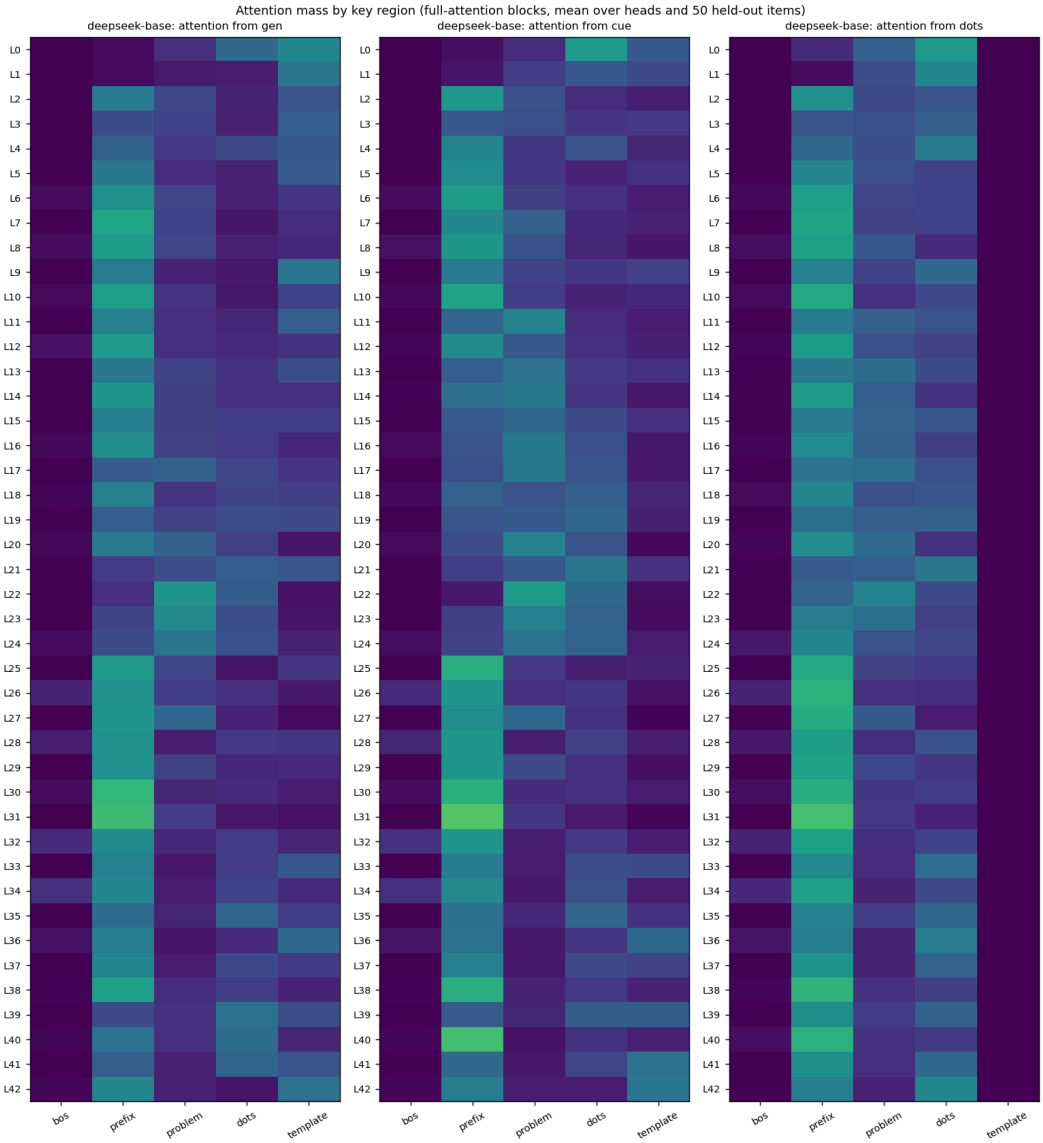

Figure 9: DeepSeek-V4-Flash-Base: attention mass by key region at every block, mean over 64 heads and 50 held-out items. Compare the chat model's figure in Part V.

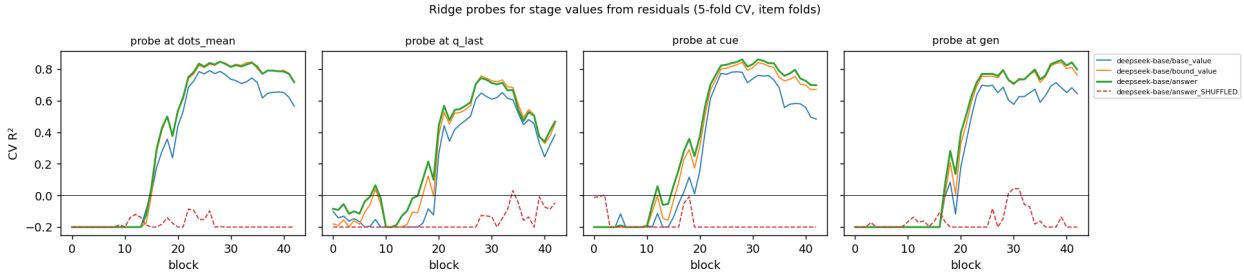

Figure 10: DeepSeek-V4-Flash-Base: ridge-probe R^2 by block for the stage values, from the dots, the last question token, the answer cue, and the generation position.

9 Part VII: what fills the span, and when the model decides to use it

A second day on the same box, with three questions. Does the filler have to be dots? Is there a point in the span where the model "changes thought"? And what, in the context, makes the chat model stop computing at the question token? Everything here uses the same 50 released items for behavior and the same 50 held-out items at $k = 50$ for dumps, in chat rendering unless stated (`reports/filler-types-and-cosine.md` is the running log).

9.1 Letters, numbers, and their scrambles

`src/jlens_filler/prompts.py` renders five filler types: dots, the alphabet, a fixed scrambled alphabet, counting numbers, and a fixed scrambled sequence of numbers. Numbers cost two tokens each on DeepSeek's tokenizer (a standalone space precedes each), so 50 numbers are 99 tokens.

Filler	Model	k=0	5	10	25	50	100	helped/hurt @50	pre-q. @50
dots	chat	35	45	42	43	49	49	14/0	35
alphabet	chat	34	41	43	41	42	43	12/4	34
alphabet, scrambled	chat	34	38	32	38	41	46	10/3	36
counting	chat	34	45	45	46	49	48	15/0	35
counting, scrambled	chat	34	31	41	41	49	50	15/0	28
dots	base	48	48	49	49	50	49	2/0	49
any other	base	48	49–50	49–50	48–50	49–50	49–50	$\leq$2/1	—

Table 11: Correct out of 50 by filler type and count (post-question), with the pre-question control at 50.

Numbers work as well as dots, in any order. Letters help less (42 to 46 even at 100 tokens, so it is not token count), in any order. Scrambled numbers hurt at $k = 5$ before helping. Every type is placement-specific: before the question, nothing helps and scrambled numbers hurt. The base is at ceiling for every type.

Model	Filler	gen→filler	cos same pos	R^2 ans|filler	R^2 ans|q_last	cent. adj.	correct @50
chat	dots	0.164	0.78	0.87	0.40	0.12	49
chat	alphabet	0.126	0.82	0.84	0.24	0.05	42
chat	alphabet, scrambled	0.113	0.85	0.83	0.43	0.03	41
chat	counting	0.227	0.77	0.89	0.51	0.04	49
chat	counting, scrambled	0.231	0.77	0.91	0.52	0.01	49
base	dots	0.203	0.64	0.85	0.74	0.11	50
base	alphabet	0.180	0.77	0.83	0.88	0.04	50
base	alphabet, scrambled	0.164	0.81	0.78	0.87	0.02	50
base	counting	0.271	0.72	0.82	0.78	0.03	49
base	counting, scrambled	0.267	0.72	0.88	0.76	0.00	50

Table 12: Anatomy by filler type (50 held-out items, $k = 50$ items). Attention is the generation position's mass on the filler over the last third of blocks.

Attention on the filler tracks the chat model's gain: letters 0.11 to 0.13 and 41 to 42 correct, dots 0.16 and 49, numbers 0.23 and 49. The answer is decodable from every filler type at 0.78 to 0.91 in both checkpoints, so every type ends up holding the answer; what differs is how hard the answer positions read it. The base reads every type harder than the chat model, in the same order, so which token kind attracts the answer positions is pretrained. The two checkpoints separate in one column only, the answer at the question token, and they separate there for all five fillers.

The same fillers on Qwen3.5-9B: the untrained model is at floor for all of them, and the dots-only LoRA gets nothing from letters (42 → 40) and is hurt by numbers (42 → 37; scrambled, 42 → 30

with 12 items hurt and none helped). Its answer position, which ignores dots and letters (attention 0.01), attends to filler digits at 0.05 with single heads at 0.5: a model that does its arithmetic at the answer position reads filler digits as problem digits. DeepSeek's chat model has the opposite relation to the same tokens.

9.2 Adjacent-position cosine: no change of thought

For each item, the residual at every filler position and every block. Adjacent cosine is $\cos(h_p, h_{p+1})$ per block and flattened over blocks; the *item-centered* version subtracts the per-position mean over items first, removing the position-identity component that dominates filler residuals. Change points are boundaries where an item's flattened centered series drops more than two standard deviations below its own mean (`scripts/analyze_filler_cosine.py`).

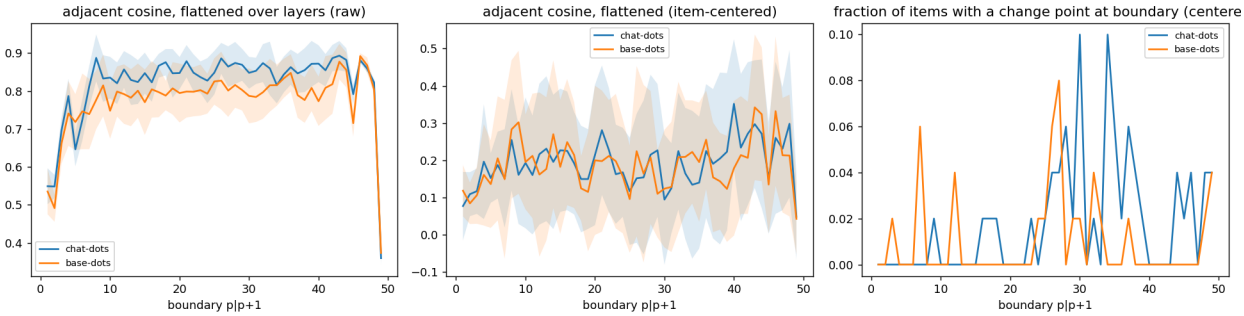

Figure 11: Adjacent-position cosine across fifty dots, chat versus base: flattened over blocks (left), item-centered (middle), and the fraction of items with a change point at each boundary (right).

Model (dots)	centered adjacent cosine, late blocks	change pts/item	where
Qwen3.5-9B base	0.25 to 0.30	0.15	diffuse
Qwen3.5-9B $k = 0$-only	0.44 to 0.51	0.69	F1/2 (42 of 100 items)
Qwen3.5-9B dots-only	0.75 to 0.82	4.2	F2/3, F3/4 (100/100), F7/8 (98)
DeepSeek chat	0.06 to 0.19	0.88	diffuse, none above 5 of 50
DeepSeek base	0.06 to 0.19	0.48	diffuse, none above 4 of 50

Table 13: Problem-specific content carried from one dot to the next.

Three findings. The raw series has exactly three drops in every item and every model, at F1|F2, F2|F3 and F49|F50, and all are tokenization (the first dot has no leading space, the last merges with the newline); with letters the one shared drop is the z→a wrap; with numbers the series alternates because the tokens alternate number, space. Between artifacts the raw adjacent cosine is 0.7 to 0.96 and flat. Second, after centering, DeepSeek's adjacent cosine is 0.01 to 0.19 for every filler type: each position's problem-specific content is nearly unrelated to its neighbor's, and change points are scattered. The span is fifty weakly related snapshots, not a trajectory with segments. Third, the trained Qwen that ignores its dots carries one constant problem vector along the span (0.77 after centering) with a settling transient in the first ten dots identical across all 100 items; that signature is specific to identical tokens (it falls to 0.08 to 0.23 with letters or numbers), so redundancy has to be compared within a filler type, and within dots the contrast with DeepSeek stands. One caveat: the item-centered cosine between filler and answer positions rises along the span in the chat model

and falls in the base, but it also rises in the Qwen dots-only model, so that gradient alone is not evidence of use.

9.3 The announcement, not the tokens, moves the computation

The 0.74 versus 0.40 question-token probe of Part VI was measured with dots present. With no filler anywhere in the prompt ($k = 0$: no filler sentence, none in the demonstrations, none in the target), both checkpoints hold the answer at the last question token at R^2 0.84. The question token precedes the dots, so the dots cannot cause the gap; what differs between $k = 0$ and $k = 50$ is the system sentence announcing fifty filler tokens and the demonstrations containing them. So I rendered the sentence and the demonstrations with N dots and the target with none (`extract_dsv4.py --announce-filler N`).

Announced dots, none delivered	0	5	25	50	50, delivered
chat: R^2 answer at question token	0.84	0.51	0.36	0.31	0.36
base: R^2 answer at question token	0.84	0.86	0.85	0.55	0.65
chat: correct of 50	34			32	49
base: correct of 50	48			46	50

Table 14: Deferral is triggered by the announcement and graded by the announced count. Answer probes at the cue and generation positions stay at 0.79 to 0.83 in every condition.

The chat model reads the number in the instruction and defers in proportion: five promised dots halve its in-place computation, twenty-five bring it to the fifty-dot level, and the value is then assembled at the cue and generation positions instead (0.8 there in every condition). The base ignores the announcement until it is fifty, and defers partly. Accuracy moves much less than the probe (chat 34 to 32), so deferral is not itself the $k = 0$ deficit; the deficit is present at 0.84 and at 0.31 alike, and no linear probe at any position sees it. What the dots do is give a model that has planned to finish in the span a span to finish in: promised and delivered, 49; promised and absent, 32.

Which channel carries it. The announced-but-absent prompt announces in two places: the system sentence and the five demonstrations, each with fifty dots between question and answer. Separating them (`announce_mode`) gives a double dissociation.

Target $k = 0$, announced by	chat: q_last probe	chat: correct	base: q_last probe	base: correct
nothing	0.84	34	0.84	48
sentence only	0.83	28	0.84	50
demonstrations only	0.40	34	0.55	46
both	0.31	32	0.55	46

Table 15: The demonstrations move the computation off the question token; the sentence costs the chat model accuracy without moving it.

With fifty dots delivered in the target, the same split: nothing announced 34 → 39, sentence only 28 → 36, demonstrations only 34 → 47 (13 helped, 0 hurt), both 34 → 49. Unannounced dots help a little; demonstrations of a span being used make them work; the sentence adds two items on top.

So the deferral is in-context format learning: five examples of question, span, answer teach both checkpoints, the chat model more strongly, to finish the arithmetic after the question rather than at it, in proportion to the span the examples show. The instruction sentence does something different. It costs the post-trained model six items (34 to 28, its worst score in any condition) without changing where the answer is computed, and it does nothing to the base.

Where the sentence is read. Attributing attention keys to the 21-token filler sentence in the system message (a sixth region in `dump_dot_attention_dsv4.py`): from the last question token, the chat model puts 0.04 of its attention on that sentence in blocks 20 to 24 (single heads at 0.16), 0.06 in blocks 26 to 38 (a head at 0.25), and 0.17 at block 40 (a head at 0.38), against a uniform-attention level of 0.026; the base stays at or below 0.009 through block 36 and reads it at 0.08 at block 40. The numbers are identical whether the dots are delivered or not. The sentence sits about 600 tokens before the question, outside the sliding window, so this is reading through the compressed keys in the even blocks, and its onset at block 20 is where the question-token probe first separates between announced and unannounced prompts. Post-training installed heads at the question token that read the filler instruction. Given the channel decomposition, they are not the deferral mechanism (the demonstrations do that); they are the chat model's route to a sentence whose presence costs it accuracy. What the demonstrations do to the question token is not located: under demonstrations-only announcement the question token's attention on the demonstrations' filler spans is at or below their share of the keys in both checkpoints (0.16 chat, 0.21 base, against about 0.32), so it is not a matter of reading those spans.

Two more pieces. The first-token distribution (from the sweep files) is three times flatter in the chat model at $k = 0$ than in the base (top-10 entropy 0.76 versus 0.30 nats; top-1 probability 0.74 versus 0.89), sharpens to the base's level with fifty dots (0.26), flattens further when filler is announced and absent (1.13), and on every miss has the correct number at median rank 3. And in plain rendering, without turn markers or the think tag, the chat model drops to 19/50 at $k = 0$ and still climbs to 44 at $k = 100$ with dots: the chat template was helping, and the dot effect survives a change of rendering.

10 Why does DeepSeek differ?

Part VI answers the headline question: post-training, and Part VII says how: shown examples with a span, the post-trained model defers its computation into the span it expects, and the instruction announcing the span degrades its answer. What follows is the reasoning that stood before that run, kept because it says which explanations the earlier evidence had already weakened.

Scale. Weakened. DeepSeek V4 Flash runs about 13B active parameters (256 experts, 6 active plus one shared, 43 blocks, width 4096), the same regime as the models tested. Total parameters (roughly 280B) is the only scale axis left, and it is confounded with everything else. The trained 9B is the sharper counterexample: it reached DeepSeek-level accuracy at $k = 0$ (72% versus 70%) and got nothing from dots. Nicole's interpretation was that filler amplifies a circuit the model nearly has. The 9B nearly had the circuit and no amplification happened.

Architecture. Partly ruled out. Two 3B-active mixture-of-experts models behave like the dense ones, so expert routing alone is not it. Three features unique to V4 among everything tested remain: the four-stream hyper-connection residual (four lanes per position to hold a value in without overwriting), multi-head latent attention, and a learned top-512 sparse attention index.

The residual width is my candidate for why the habit *works well* once a model has it: writing a value into another position without clobbering what is there is cheap with four lanes and expensive with one, and the $k = 0$-only model shows the single-stream failure mode directly.

Post-training. Confirmed by Part VI as the source of the *deficit*, though not of the dot-reading anatomy, which the base already has. Using dots means writing intermediate state into positions after the question and reading it back at the answer. No gradient in my training ever rewarded that, because the direct route always worked, and the learner takes the path that already exists. This matches Pfau et al. (2024), who found filler helps only when training makes it useful. DeepSeek V4 was trained hard with reinforcement learning to reason across a span of tokens before answering; a model that has internalized "the span between question and answer is where thinking happens" would plausibly treat any trailing tokens as thinking room. Nicole's readouts look like that: values staged by depth, broadcast across the span, read from the late band right before the answer. The awkward fact is that Qwen3.5 also has a thinking mode and shows nothing, so the difference would have to be in how each lab trained its non-thinking mode. Part V adds that the result is visible in attention: DeepSeek's answer positions read the dots at 16 to 43% while every Qwen variant, trained or not, reads them at 1 to 4%. Part VI then shows the base reads them the same way, so the reading is pretrained and what post-training installed is the deficit that makes the reading matter.

The experiment that decided. The base run in Part VI: flat at 94 to 100% with or without dots, in either placement. Post-training built the deficit; the repair uses a dot-reading anatomy the base already has. The open question is now what in DeepSeek's post-training does this, and the cheap partial check remains DeepSeek-V2-Lite-Chat (2.4B active, no hyper-connections) on the existing harness.

11 Limitations

- Per-cell lens readouts on Qwen rank the first digit of a value, because the tokenizer splits digits. The derangement controls bound how much this could hide, and the probes and lesions do not depend on it, but a whole-number readout (Llama, DeepSeek) would be cleaner.
- Linear probes cannot separate stored inputs from computed outputs on an affine task. The probe results say the dots encode the relevant numbers; they do not say the dots compute with them.
- Twenty-four of thirty-two Qwen3.5 blocks are linear attention. Attention results cover the other eight. In the recurrent blocks the dots are recurrence steps regardless of content, so a structural "extra compute" from dots exists that the lesions remove the content of but not the steps.
- The training phase shows that answer-only supervised fine-tuning with dots present does not induce workspace use in Qwen3.5 up to 9B. It does not show that no recipe can. Dense supervision on intermediate values placed in the dots, a task where a single forward pass provably cannot fit the computation, and reinforcement learning with a compute penalty are all untested.
- The Llama weights are an ungated mirror (`unsloth/Llama-3.1-8B-Instruct`) of the gated Meta checkpoint. This does not affect behavioral numbers.
- Fifty items is a small screen. The log-probability endpoint, the 100-item held-out sets, and the 200-item replication of the one bump that appeared are what make me confident in the

nulls.

12 What I would do next

In order of information per GPU-hour:

1. Patch the base's question-token residual into the chat model under an announced span, and the chat model's announced-span question token into its own unannounced run, to test whether deferral is carried by that one position. The announcement's footprint is now located (blocks 20 to 40 at the question token); the next step is to ablate those heads' attention to the sentence and see whether the chat model's in-place computation and its $k = 0$ accuracy return.
2. Run DeepSeek-V2-Lite-Chat on the existing harness. Twenty minutes. Tests whether the DeepSeek training pipeline produces the effect without hyper-connections.
3. Train a model where dots are *necessary*: a chain long enough that the direct computation cannot fit, with the loss also applied to intermediate values written at the dot positions during a first phase, then removed. If that produces a dot-dependent model, Nicole's causal ladder can be run on it and compared with DeepSeek's mechanism.
4. On the DeepSeek side, read the four residual streams separately. The unresolved question in Nicole's pipeline is how the released lens collapses four streams to one; values living in one lane may be invisible after the collapse, and her "raw first product never appears" result could be that artifact.

A Reproducibility

All code is in the project repository. Model-touching scripts run on one GPU with `transformers` from main (Qwen3.5 requires it), `peft`, and PyTorch 2.11 with CUDA 12.8.

- Behavioral sweep: `scripts/extract_hf.py --phase eval` on any `configs/*_dot_length_sweep.json`; `scripts/analyze_behavior_sweep.py` for tables; `scripts/screen_models.sh` to loop models.
- Data and training: `scripts/build_varbind_sft_data.py` (`--chain-len`), `scripts/build_onestep_varbind_configs.py`, `scripts/train_varbind_lora.py` (`--ks` selects the dot counts).
- Readouts: `scripts/extract_hf.py --phase filler --adapter`, `scripts/build_artifacts.py` for viewers, `scripts/analyze_hf_lens_grids.py` and `_deep.py`, `scripts/select_heldout_lens_cases.py`.
- Causal: `scripts/build_varbind_counterfactuals_hf.py`, `scripts/patch_varbind_hf.py` (single-cell grid; `--lesion-all-dots [--lesion-all-layers]`).
- Anatomy: `scripts/dump_dot_residuals_hf.py`, `scripts/dump_dot_residuals_dsv4.py`, `scripts/dump_dot_attention_dsv4.py`, `scripts/setup_dsv4_box.sh`, `scripts/run_dsv4_dot_dump.sh`, `scripts/run_dsv4_base_eval.sh`, `scripts/run_dsv4_base_dot_dump.sh`, `scripts/run_dsv4_filler_types.sh`, `scripts/run_hf_filler_types.sh`, `scripts/run_dsv4_followups.sh`, `scripts/run_dsv4_announce.sh`, `scripts/analyze_filler_cosine.py`, `scripts/summarize_filler_types.py`, `scripts/probe_k0_dump.py`, `scripts/analyze_dot_residuals.py`, `scripts/probe_extra_targets.py`, `scripts/plot_dot_analysis.py`.
- Results: `results/<model>/` per model; training under `results/qwen3.5-9b/lora-*`; lens grids under `results/qwen3.5-9b/lens-heldout-k50/`; lesions and patching under `results/`

`qwen3.5-9b/lora-*/lesion-all-layers/` and `results/qwen3.5-9b/patch-heldout-0067/`; anatomy under `results/qwen3.5-9b/dot-dump/analysis/`. The 1.3 GB residual dumps stay on the compute box.

- Nicole's DeepSeek pipeline and results: `scripts/extract_dsv4.py` and `reports/algorithm-exploration-findings.md`.